

SIH 2026 — Problem Statement Documentation

PS ID: SIH26170 | AI-Driven Anomaly Detection in Component Burn-In & Screening

1. Problem Statement Snapshot

Field	Detail
PS ID	SIH26170
Title	AI-Driven Anomaly Detection in Component Burn-In & Screening
Organization	Indian Space Research Organisation (ISRO) / Department of Space
Theme	Smart Automation
Category	Software
Domain	Aerospace reliability engineering, predictive quality assurance, applied ML on time-series sensor data

 **Note on sourcing:** SIH does not publish PS text as a single stable public API — the official copy lives only inside the SIH portal (sih.gov.in) behind a login-gated dashboard for registered teams. This document reconstructs the PS accurately from the officially released problem statement text as mirrored on multiple SIH 2026 archive sites, cross-checked with each other. **Before you finalize your approach, log into sih.gov.in with your team credentials and copy the exact PS wording (background, description, expected solution, and any attachment links) — wording nuances matter during evaluation.**

2. Understanding the Problem

2.1 Background (industry context)

High-reliability electronic components used in satellites and launch vehicles must survive years in orbit with **zero possibility of physical repair**. To catch defects before flight, ISRO subjects every batch of flight-grade components to **Environmental Stress Screening (ESS)**, of which **Burn-In testing** is the core step:

- Components are operated continuously at elevated temperature (commonly **125°C**) for an extended duration — typically **168 hours (7 days)**.

- At fixed checkpoints during this stress window (e.g., **0h, 24h, 96h, 168h**), the test equipment measures parametric values such as:
 - **Iddq / standby leakage current**
 - **Leakage current (μA)**
 - **Propagation delay**
- After burn-in, each component is passed/failed against a **static datasheet limit** — e.g., "reject if leakage > 50 μA ."

2.2 The core problem

Static, absolute limits only catch components that are *already* broken. They miss a more dangerous class of defect:

A component can stay **within** the datasheet limit (e.g., 45 μA when the limit is 50 μA) while still being **abnormal relative to the rest of its manufacturing lot** (whose average might be 10 μA) — or it may be **drifting** upward over time in a way that predicts it will fail *after* it reaches orbit.

These "latent defects" pass conventional screening and only reveal themselves as in-orbit field failures — which, for space hardware, are catastrophic and unreparable.

2.3 What ISRO is asking teams to build

The PS asks for a predictive ML system, generally structured as **two modules**:

Module A — Dynamic (Lot-Relative) Outlier Detection

- Replace the fixed pass/fail cutoff with a *relative* one.
- For each production lot/batch, compute the lot's own statistical distribution (mean, std-dev, or robust equivalents like median/IQR).
- Flag a component if it deviates significantly from its **own lot's** behavior — even if it is still inside the absolute datasheet limit.
- Typical techniques: Z-score / robust Z-score (median + MAD or IQR-scaling to resist outlier skew), Isolation Forest, One-Class SVM, or DBSCAN-based clustering per lot/wafer.

Module B — Time-Series Drift Predictor

- Using only the **early** measurements (e.g., Value at 0h and 24h, and the velocity/slope between them), forecast the **168h (end-of-burn-in) value**.
- This lets ISRO identify a likely failure *early*, without waiting the full 7 days, saving test time and enabling faster corrective action.

- Typical techniques: Linear/Ridge/Lasso regression, Gradient Boosting (XGBoost/LightGBM), simple RNN/LSTM if you have longer time-series, or physics-informed extrapolation (e.g., Arrhenius-based degradation curves).

Common cross-cutting expectations across teams' interpretations of the "Expected Solution" section:

- Output a clear, explainable **PASS / REVIEW / REJECT** decision per component (not a black-box score).
- Provide traceability: which lot, which wafer, which parameter triggered the flag, and why (explainability — e.g., SHAP values or rule-based justification).
- Should work with **offline/air-gapped** deployment in mind (ISRO test labs are typically isolated from the internet for security).
- A dashboard/UI for QA engineers to visualize drift trends, lot comparisons, and review flagged components.
- Should be robust to noisy/low-resolution sensor data and to lots with very few samples.

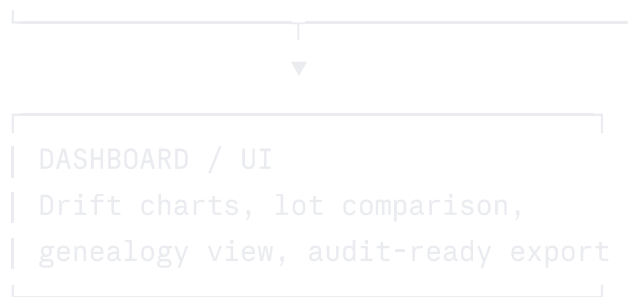
2.4 Reframe the problem in one line

"Instead of asking *'Is this component inside the datasheet limit?'*, build a system that asks *'Is this component behaving normally relative to its own production lot, and is it drifting toward failure?'* — and explain the answer to a QA engineer."

3. Solution Approach

3.1 High-level architecture





3.2 Recommended tech stack

Layer	Suggested tools
Data processing	Python, Pandas, NumPy
ML — anomaly detection	Scikit-learn (IsolationForest, LocalOutlierFactor, EllipticEnvelope), PyOD library
ML — drift prediction	Scikit-learn (Ridge/Lasso), XGBoost/LightGBM, or PyTorch/TensorFlow if using LSTM
Explainability	SHAP, LIME
Backend/API	FastAPI or Flask
Frontend/dashboard	React + Recharts/Plotly, or Streamlit for a fast prototype
Database	SQLite (for offline demo) or PostgreSQL (for scale)
Deployment	Docker / docker-compose (important — many teams highlight offline/air-gapped capability since this is for ISRO facilities)
Reporting	Auto-generated PDF/CSV audit reports (ReportLab, WeasyPrint)

3.3 Step-by-step build plan

1. **Understand & lock the exact PS text** from sih.gov.in (background, description, expected solution, any dataset link provided).
2. **Acquire/generate a representative dataset** (see Section 4).
3. **EDA**: plot leakage/Iddq/delay trajectories per lot; visualize how "in-spec but lot-anomalous" components look different from truly normal ones.
4. **Feature engineering**: per-lot mean/median/IQR, per-component deviation from lot baseline, slope ($\Delta\text{value}/\Delta\text{time}$) between 0h–24h, acceleration (change in slope), if multiple parameters — combine into a composite risk feature.

5. **Module A (anomaly detection):**

- Baseline: robust Z-score / Modified Z-score per lot.
- ML: Isolation Forest, trained per lot or on lot-normalized features globally.
- Validate against injected synthetic anomalies (see below) to measure recall.

6. **Module B (drift prediction):**

- Regression model taking (Value_0h, Value_24h, slope) → predicted Value_168h.
- Evaluate with MAE/RMSE; benchmark against a naive linear extrapolation baseline.

7. **Decision engine:** combine Module A anomaly flag + Module B predicted-limit-breach into PASS/REVIEW/REJECT, with a deterministic hard-threshold safety net so you never silently miss an obvious failure (teams commonly emphasize "zero false negatives" as a design goal, since missing a bad part is far costlier than a false alarm).

8. **Explainability:** attach SHAP values or simple rule explanations ("flagged because leakage is 3.2σ above lot mean") to every REVIEW/REJECT decision.

9. **UI/Dashboard:** batch upload → results table → drill-down chart per component → lot comparison view → exportable report.

10. **Test with edge cases:** lots with only 1-2 components, missing timestamps, sensor noise/quantization, multiple parameters simultaneously drifting.

11. **Package for offline demo:** Docker Compose so judges can run it without internet.

12. **Prepare the narrative:** quantify impact — e.g., "reduces escaped latent defects by X%, saves Y% of burn-in floor time via early prediction."

3.4 Differentiators that tend to score well (based on how past/parallel teams approached this exact PS)

- **Explainability-first UI** — judges are engineers; a black-box anomaly score is less convincing than a clear "why" per flagged part.
- **Lot/wafer genealogy tracing** — linking an anomaly back to a specific wafer/lot/equipment run to find a systemic manufacturing issue, not just a one-off part.
- **Equipment-fault vs. component-fault discrimination** — distinguishing "this component is bad" from "this test chamber/sensor is misbehaving" (a chamber-level anomaly should not fail every part in it).
- **Physics-informed constraints** — e.g., using known degradation physics (Arrhenius acceleration, TDDDB, electromigration trends) to sanity-check purely statistical predictions.
- **Deterministic safety override** on top of ML, so the system never trusts the model blindly for a false PASS.

- **Air-gapped/offline deployability**, since ISRO test floors are typically not internet-connected.
-

4. Datasets — What You Need and Where to Get Them

4.1 The hard truth about data for this PS

ISRO's actual burn-in/ESS telemetry is **proprietary and classified/restricted** — it will not be released publicly, and the PS itself does not usually ship a ready-made public dataset link. Nearly every team that has worked this problem statement has therefore taken one of two routes:

Route 1 — Synthetic data generation (most common, and generally acceptable/expected)

Build a Python generator that creates realistic burn-in telemetry:

- Simulate N lots, each with M components.
- Each component gets a baseline leakage/Iddq/delay value drawn from a lot-specific normal distribution.
- Model degradation over time (0h→24h→96h→168h) using an exponential or linear drift function with random noise.
- Deliberately inject a known number of "anomalous" components (some in-spec-but-lot-anomalous, some drifting fast, some equipment-fault correlated) so you have **ground truth** to measure your model's precision/recall against.
- This is exactly what most reference implementations for this PS do (e.g., `generate_data.py` scripts producing an `isro_burn_in_dataset.csv`), and it's a fully legitimate approach for a hackathon prototype — just be explicit in your submission that the dataset is synthetic/demonstration data, not real ISRO data.

Route 2 — Public proxy datasets on real component degradation Use real (but non-ISRO) semiconductor degradation/reliability datasets as a stand-in to prove your pipeline works on real physical measurements, then layer the synthetic ISRO-style burn-in generator on top for the exact use case:

Dataset	What it contains	Where to find
NASA PCoE — MOSFET Thermal Overstress Aging Dataset	Real MOSFET degradation data ($R_{ds(on)} = V_{ds}/I_d$ measured over aging cycles) — a good physical stand-in for "leakage/parametric drift over stress time"	NASA Prognostics Data Repository: https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/ (search "MOSFET Thermal Overstress")
NASA PCoE — IGBT/Power Electronics Accelerated Aging datasets	Similar accelerated-aging parametric data for power semiconductor devices	Same NASA PCoE repository
NASA PCoE — Bearing / Li-ion Battery degradation datasets	Not component-electrical, but useful for practicing "predict end-of-life from early degradation curve" methodology (transferable technique for Module B)	Same NASA PCoE repository
Kaggle — Semiconductor Manufacturing / SECOM dataset	Real wafer-level sensor readings from a semiconductor fab with pass/fail labels — good for practicing lot/wafer-relative anomaly detection at scale	https://www.kaggle.com/datasets/paresh2047/uci-semcom (UCI SECOM mirror)
UCI Machine Learning Repository — SECOM Data Set	Same SECOM dataset, original source	https://archive.ics.uci.edu/dataset/179/secom

Dataset	What it contains	Where to find
Kaggle — general "IC/Chip reliability/burn-in" simulation notebooks	Search Kaggle for community-generated burn-in/ESS simulation datasets for inspiration on schema design	https://www.kaggle.com/search?q=burn-in+semiconductor
JEDEC / JESD standards (reference only, not raw data)	Industry-standard definitions of burn-in, ESS, and screening test methodologies — useful for grounding your assumptions in real qualification standards	https://www.jedec.org/standards-documents (e.g., JESD22, JESD47)

💡 **Recommended hybrid approach:** Train/validate your **methodology** (feature engineering, robust scaling, Isolation Forest tuning) on the real SECOM or NASA MOSFET data to prove it generalizes to genuine physical sensor drift, then run your final demo on your own synthetic ISRO-style burn-in dataset that matches the PS's exact described columns (Component_ID, Lot_ID, Wafer_ID, Parameter, Value_0h, Value_24h, Value_96h, Value_168h, Datasheet_Limit). This gives you both scientific credibility and a demo that visually maps 1:1 onto the PS's stated scenario.

4.2 Suggested schema for your generated/working dataset

Column	Type	Description
Component_ID	string	Unique ID per physical part
Lot_ID	string	Manufacturing batch/lot identifier
Wafer_ID	string (optional)	Wafer within the lot, for genealogy tracing
Parameter	categorical	Iddq / Leakage_Current / Propagation_Delay
Value_0h, Value_24h, Value_96h, Value_168h	float	Measurements at each checkpoint
Datasheet_Limit	float	Absolute static pass/fail threshold
Chamber_ID / Equipment_ID	string (optional)	For equipment-fault discrimination
Ground_Truth_Label	categorical (synthetic only)	Normal / Lot-Anomalous / Drifting-Failure / Equipment-Fault — used only for evaluating your model, not fed into it

4.3 Data generation checklist

- ☐ Simulate ≥ 10 –20 lots with 50–200 components each (to get meaningful lot statistics).
- ☐ Vary lot baselines slightly (real manufacturing lots aren't identical) — this is what makes the "dynamic" (lot-relative) requirement meaningful.
- ☐ Inject 3 anomaly archetypes: (a) high-but-still-in-spec outlier, (b) fast-drifting-toward-168h-failure, (c) whole-chamber/equipment correlated anomaly.
- ☐ Add sensor noise/quantization to mimic real ATE resolution limits.
- ☐ Keep a hidden "ground truth" label set aside purely for computing your own precision/recall/F1 to quote in your presentation.

5. Evaluation Metrics to Track (for your own validation and for the judges)

- Module A (anomaly detection):** Precision, Recall, F1-score, ROC-AUC against injected ground-truth anomalies; false-negative rate is the most important number to report (missing a real defect is the costliest failure mode for this domain).

- **Module B (drift prediction):** MAE, RMSE of predicted vs. actual 168h value; report how much test-floor time could theoretically be saved by trusting an early (e.g., 24h) prediction instead of waiting the full 168h.
 - **Operational metric:** number of components correctly flagged that a static-limit-only approach would have missed — this is the single most persuasive number for your pitch, since it directly demonstrates the PS's core motivation.
-

6. Suggested Presentation Narrative (for PPT/pitch)

1. Problem: static limits miss latent, lot-relative, and drift-based defects → in-orbit failures.
 2. Approach: Module A (lot-relative anomaly detection) + Module B (early drift prediction) + explainable decision engine.
 3. Demo: upload a batch → dashboard shows flagged components with reasons → drill into lot/wafer genealogy.
 4. Impact: quantify defects caught that static limits would have missed; quantify test-time saved by early prediction.
 5. Responsible-AI framing: emphasize this is a **decision-support** tool for QA engineers, not an autonomous accept/reject authority — this framing is consistent with how ISRO would realistically want such a tool positioned in a safety-critical pipeline.
-

7. Key References

- Official PS portal (verify exact wording before finalizing): <https://sih.gov.in/sih2026PS>
 - NASA Prognostics Center of Excellence Data Repository:
<https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>
 - UCI SECOM semiconductor manufacturing dataset:
<https://archive.ics.uci.edu/dataset/179/secom>
 - JEDEC reliability/burn-in standards: <https://www.jedec.org/standards-documents>
 - SHAP documentation (for explainability layer): <https://shap.readthedocs.io/>
 - PyOD (Python Outlier Detection library, useful for Module A):
<https://pyod.readthedocs.io/>
-

Prepared as a working reference for SIH 2026 internal-hackathon and national-round preparation. Cross-verify all PS wording against the official sih.gov.in portal before submission, since PS descriptions can be revised by the sponsoring organization.